

Saliency-Guided Burst Denoising for Low-Light Photography

Maximilian Dittgen
myd4@cornell.edu
Cornell University
Ithaca, New York, USA

Justin Kong
jk2338@cornell.edu
Cornell University
Ithaca, New York, USA

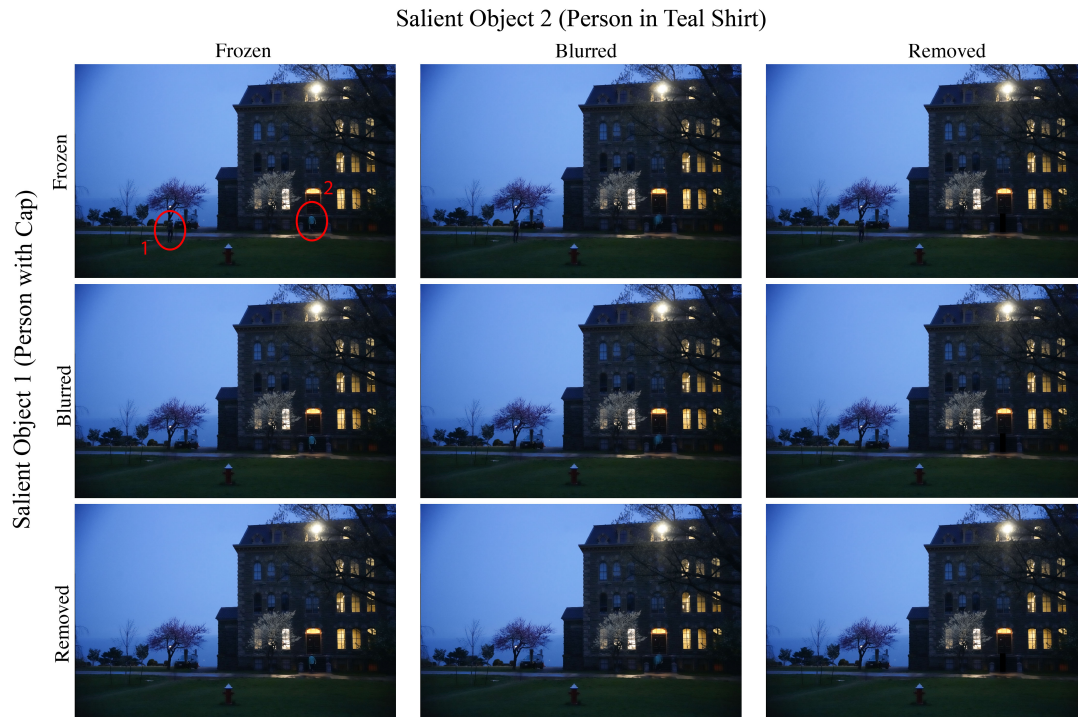


Figure 1: Several of the ways a low-light image can be edited using our burst stacking pipeline. This image contains two people in motion as our salient objects. Each person's motion can be frozen, blurred, or removed altogether independently of each other depending on the photographer's preference.

Abstract

Camera sensors gather less photons in low-light environments, causing noisy images and providing a challenge for photographers looking to capture high-fidelity low-light images. Smartphone cameras mitigate noise by capturing bursts of images then using stack-and-align pipelines to smooth the scene. However, moving objects in a scene will introduce motion blur in the composited burst, making it difficult for smartphone photographers to exercise a high degree of creative control when shooting in dark settings. In this work, we introduce a burst imaging pipeline and editing suite that

allows photographers to modify the appearance of moving objects after capturing a low-light burst. We leverage real-time object detection models (YOLO) to detect moving, salient objects throughout an image burst and perform saliency-aware denoising. We then implement an interactive application that allows users to pass their bursts through our pipeline before exploring several possible appearances of any moving object in their scene. This platform solves issues of motion blur in low-light burst photography while creating an interactive sandbox for editors to explore a fun space of possible images.

CCS Concepts

• **Computing methodologies** → **Computational photography**;
Image processing.

Keywords

Motion Blur Removal, Image Denoising, Salient Object Detection

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Maximilian Dittgen and Justin Kong. 2018. Saliency-Guided Burst Denoising for Low-Light Photography. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Low-light photography poses a challenge in computational imaging. Due to the limited photon count in dark scenes, attempting to amplify the weak signal captured by an image sensor results in low signal-to-noise ratio images that are grainy, noisy, and lack high-fidelity detail. Many traditional image denoising techniques that aim to remove noise from images struggle in low light settings since the noise tends to overwhelm the low light signals, making image recovery a difficult process [2]. Aside from amplifying a weak image signal, there are other tradeoffs photographers can make to recover information from a dark scene. A photographer may choose a fast-aperture lens with a high light intake—but such lenses are expensive and difficult to integrate onto handheld smartphone cameras. Alternatively, a photographer may opt to slow their shutter speed to aggregate sensor data over time—however, without proper image stabilization long exposures cause motion blur, especially if there is a moving object in frame. To address the trade-



Figure 2: Our goal is to enable more expressivity in handling moving objects in low-light bursts. In the left-most image we see our original stacked burst, which blurs the two cars driving through the scene. In the middle image, both cars are removed; in the right image, we choose a moment in time to freeze and sharpen the cars.

offs associated with low-light photography, modern smartphones and imaging systems adapt burst photography, capturing a rapid sequence of short-exposure high-ISO frames, aligning the frames to reduce motion blur, then averaging across frames suppress noise [4]. Aligned burst photography works well for the smartphone market. They can integrate with low-cost, high-aperture smartphone lenses, stabilize amateur hand-held photography, and simulate long exposures while retaining temporal information for each individual component frame.

While pipelines for aligned burst photography are well-defined for static scenes, low-light imaging pipelines are ill-defined for how to handle moving objects within a scene. At the moment, many imaging pipelines blur and ghost moving objects, as synonymous with the expected behavior of long-exposure photography. However, the photographer's intention of how to handle moving objects in the scene may vary. The object's motion could be accidental, in which case a photographer may want to capture the object at rest. Conversely, the object's presence in the scene may be an accidental byproduct of a long exposure, in which case the photographer may

want to remove the object from the scene altogether. At the moment, there exists no tool for photographers to interactively edit the appearance of moving objects in low-light bursts.

Recent approaches for freezing or removing moving objects in low-light bursts leverage motion alignment, optical flow, or learning-based methods [6, 7]. However, these approaches often fail to provide fine-grained control over individual salient objects, especially when users wish to manipulate or remove objects for aesthetic or practical purposes. Additionally, many approaches currently fail to automate the process of identifying salient objects that a photographer may wish to manipulate. In this paper, we propose a system that enhances the expressibility of traditional burst denoising pipelines by providing users an interactive suite to edit how moving objects are treated in low-light smartphone photographs.

We make the key observation that moving objects in scenes, such as people or animals, tend to be salient. Thus, we propose a pipeline that leverages real-time salient-object detection to extract salient moving objects from low-light bursts, then process them differently than the rest of the image depending on the photographer's intentions. Specifically, we leverage the real-time, single-shot object detection model YOLOv11 (You Only Look Once v11) at the core of our pipeline to identify salient objects to present to the photographer for editing [10].

By combining homography-based alignment, moving-object detection, image stacking, and interactive saliency control, our method enables the selective preservation or removal of dynamic objects, producing cleaner and sharper reconstructions in complex scenes. This approach is especially valuable for photographers aiming to eliminate motion blur from burst-captured images while retaining control over key visual elements.

As an example, suppose that a landscape photographer is capturing a low-light burst of a scene when a bird flies through frame. With a traditional burst alignment, the bird's movement would cause motion blur and ghosting effects in the rendered image. Using our burst processing pipeline, the photographer can load their burst into our software, then select a static position from the bird's flight to snap its motion to. Or, if the photographer did not want the bird present in the image at all, the photographer can remove the bird, and all blurring and ghosting effects produced by the bird would be removed from the final image. What otherwise would have been a tedious and manual process can now be completely automated as a result of our pipeline.

In this paper, we will detail the algorithms that power our approach and describe the integrated software that automates the entire editing workflow. Specifically, we will cover:

- **Salient Object Detection:** Identifying moving objects within a scene, which we refer to as "salient object detection" throughout the paper.
- **Denoising Techniques:** Methods for refining the final image while accounting for the presence of salient objects.
- **Automated Manipulation:** Automating the selective manipulation of salient objects to enhance the final result.
- **Integrated Software:** The software platform that combines these features, enabling photographers to effortlessly edit their images.

2 Related Works

We summarize related works for each phase of our low-light image processing pipeline.

2.1 Aligned Bursts for Low-Light Image Denoising

Debevec and Malik introduced image compositing techniques for recovering scene information lost by photography in 1997 [1]. In this case, Debevec and Malik sought to recover HDR curves from images with shallow bit depths by capturing a scene at multiple exposure levels, then exercising dynamic range compression to extend the bit depth of each merged pixel. Such a technique is effective at pooling deep information from multiple shallow image frames, but requires taking frames at a wide range of exposure levels and running intensive computation, both of which are inaccessible to low-cost consumer cell phones.

Hasinoff et al. (2016) ported early burst compositing algorithms onto commercial Android phones through HDR+, which successfully applies a "robust merge" across a series of uniform-exposure low-light smartphone photographs to attain a denoised, sharp output [6]. Hasinoff et al. acknowledges the Poisson-distributed nature of high-ISO image noise, from which it follows that averaging across multiple aligned image frames would approximate the image signal beneath the noise. HDR+ explicitly mentions and addresses moving subjects in low-light bursts. After transforming images to the spatial-frequency domain, images are tiled and tiles in each burst image and spatially shifted to align frequencies with tiles in the image taken before and after itself. This has the effect of reducing noise and increasing sharpness for moving objects, but does not address motion blur.



(c) Temporal mean with alignment

(d) Robust merge with alignment

Figure 3: Figure adapted from Hasinoff et. al 2016 [6]. Illustrates effect of FFT-based alignment on denoising a moving object in a low-light image burst. Notice that the "robust merge" image is noticeably sharper and denoised with respect the temporal mean, but still retains motion blur.

Godard (2018) introduces Deep Burst, a tool for deep learning-driven burst denoising [5]. By combining priors from image restoration datasets and noise distributions learned from a series of input burst frames, Godard's model reliably denoises images and sharpens objects affected by motion blur beyond what HDR+ can do. Although this work demonstrates how a CNN can be a lightweight additional layer on top of a traditional burst pipeline, Deep Burst fails to grant expressibility to the photographer on how to deal with

objects in motion. Moving objects are either artificially sharpened or kept blurred depending on the CNN's output.

Tabellion et al. (2023) demonstrates the multitude artistic possibilities of mimicking long exposures and background blur using aligned and stacked bursts [11]. Thus, our research aims to combine the robust low-light image stacking from Hasinoff et al., the light-weight CNN addition from Godard, and the artistic expressibility from Tabellion.

2.2 Salient Object Detection - YOLO

Redmon et al. (2015) introduced YOLO (You Only Look Once) as a model for real-time, unified object detection. By segmenting an image into bounding boxes and running a single pass through a CNN, probabilities of objectness can be assigned to each bounding box and returned to the user.

From 2015 through 2025, 10 iterations of YOLO have been released. Each iteration focuses on improving performance and speed while maintaining a consistent level of accuracy. YOLOv1 runs at 45Hz with an accuracy of 65.5%. Depending on hardware configurations, YOLOv11 can exceed 250Hz (4ms per identification) at an accuracy ranging from 71% for traffic signs to 96% for animals [8].

We expect that most moving objects in our images will be people or animals, which comprises most of YOLOv11's training corpus. Given its speed and domain accuracy, YOLOv11 is an ideal salient object detection model for our pipeline.

3 Methods

In this section, we outline the goals of our output and describe the key features implemented in our editing suite. These features are designed to provide users with flexible control over dynamic elements in burst images:

- (1) Naive Processing: What does the result look like if we simply average or stack the burst images without accounting for motion? This serves as a baseline for comparison.
- (2) Object Removal: Can we automatically detect and remove moving objects from the scene to produce a clean, static background?
- (3) Object Splatting: If a user wants to reposition a moving object to a specific location, can we generate a plausible, high-quality image reflecting that change?

For all the features in this section, we assume the user inputs a burst of n images, which we denote $x_1, \dots, x_n$. We also assume that the background of these images don't change too much from frame to frame. A full graphic of our batch processing steps can be seen in Figure 5.

3.1 Naive Denoising

In this section, we will be talking about the naive denoising method that's typically seen in many smartphone devices. This involves taking the set of burst images, aligning the images, and then averaging those images. Note that if we don't align the images to begin with, shifts in our images (potentially due to camera motion) will be averaged together and the entire image will look blurred. By aligning the images, we're making sure that many of the high fidelity features don't get lost due to averaging.



Figure 4: Our implementation of naive denoising. Left picture was taken at 128,000 iso, 1/1600 shutter. Right picture is aligned average of 30 frames, followed by a spatial bilateral filter.

In order to align the images in our burst, we must calculate homographies between corresponding features.

Let $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^3$, where $\mathbf{x} = [x, y, 1]^T$ and $\mathbf{x}' = [x', y', 1]^T$, which represent similar features in two images that we would like to map on top of each other. The homography matrix $H \in \mathbb{R}^{3 \times 3}$ is a nonsingular matrix which represents a transformation from $\mathbf{x}$ to $\mathbf{x}'$ such that

$$\begin{bmatrix} x_{\text{new}} \\ y_{\text{new}} \\ z_{\text{new}} \end{bmatrix} = H\mathbf{x},$$

where

$$\mathbf{x}' = \frac{1}{z_{\text{new}}} \begin{bmatrix} x_{\text{new}} \\ y_{\text{new}} \\ z_{\text{new}} \end{bmatrix}.$$

This homography allows us to map features from one image to its corresponding features on a different image. Let H_{ij} represent the transformation that maps x_i to x_j . To compute this homography, we use scale-invariant feature transforms (SIFT) to get pairs of corresponding points between two images [9]. We then use RANSAC to compute the homography H_{ij} [3]. We then align all the images with each other, and then crop the image with the largest overlap.

We write the full naive denoising algorithm in Algorithm 1. At a high level, Algorithm 1 will align all the images with each other, and then crop the the image such that the cropped image has the most amount of information (the region with the most amount of frames that map to those coordinates), and then return that image. Since we are taking a burst of images, the objects that are not moving should have their noise averaged out, and the objects that are moving will leave motion blur artifacts.

One of the key insights for our paper is that objects that are moving in a scene are typically salient objects. Therefore, if we are able to detect where the salient objects are, we can crop out those regions so that we're left with the background image. We can then average out the remaining frames to get a denoised version of our background.

Most of the time (depending on application), the salient object in motion will typically be something meaningful (i.e. a person, bird, stick, etc.). Therefore, we can use an object detection model to detect the objects in each frame. This will inform us of where we can crop.

For our pipeline, we use a general pretrained YOLO model, specifically YOLOv11, imported using Ultralytics. This specific model has 81 total output classes. We constrained the model to only output a

Algorithm 1 Naive Denoising

```

1: Input burst of images  $I_1, \dots, I_n$ 
2: Initialize accumulation buffer  $\mathbf{x}_{\text{sum}}$  and count buffer  $\mathbf{c}$  with
   zeros, size  $(\tilde{H}, \tilde{W}, 3)$ 
3: for  $i = 1$  to  $n$  do
4:   if  $i \neq 1$  then
5:     Detect keypoints and compute matches between  $I_i$  and
        $I_1$  using SIFT
6:     Estimate homography  $H_{i1}$  from  $I_i$  to  $I_1$  using RANSAC
7:     Warp  $I_i$  to the reference frame:  $I'_i = \text{warp}(I_i, H_{i1})$ 
8:   else
9:     Set  $I'_i = I_1$ 
10:  end if
11:  for each pixel  $(x, y)$  in  $I'_i$  do
12:    Accumulate:  $\mathbf{x}_{\text{sum}}[x, y] += I'_i[x, y]$ 
13:    Count:  $\mathbf{c}[x, y] += 1$ 
14:  end for
15: end for
16: Compute final image:  $\mathbf{x}[x, y] = \mathbf{x}_{\text{sum}}[x, y] / \mathbf{c}[x, y]$  where
     $\mathbf{c}[x, y] > 0$ 
17: return Largest valid crop from  $\mathbf{x}$ 

```

subset of objects that move in the real world, reducing the count to 18. We specifically choose the YOLO class of models for it's small model size, and ease of finetuning for more domain specific objects in motion. This does limit the number of unique objects we are able to detect, which we talk more about in Section 5. For Algorithm 2,

Algorithm 2 Salient Object Detection & Removal

```

1: Input burst of images  $I_1, \dots, I_n$ .
2: for  $i = 1, \dots, n$  do
3:   Detect objects with YOLO, and mask out objects to get
       background images.
4: end for
5: return images inputted to Algorithm 1.

```

note that for the pixels that have been masked out, we do not increment the count in the Naive Denoising algorithm, as to not let the masked out pixels (represented with zeros) influence what the average of pixels across the burst returns. This algorithm should return the average of all background images in the burst of images. This essentially removes the moving object while denoising the background of our image.

3.2 Salient Object Splatting

In an alternate setting, suppose the photographer didn't want to completely remove the image. Let x_j be the frame where the salient object is in the desired position, as determined by the photographer. We want our final image to have the salient object in this position, but returning x_j specifically will result in an image that's very noisy. Therefore, we would like to use the other images in our burst to denoise our image.

To do this, we take our burst of images and denoise the background, as defined in Algorithm 2. Let the averaged background image be $\tilde{x}$. We can then take the cropped out object of x_j , call the

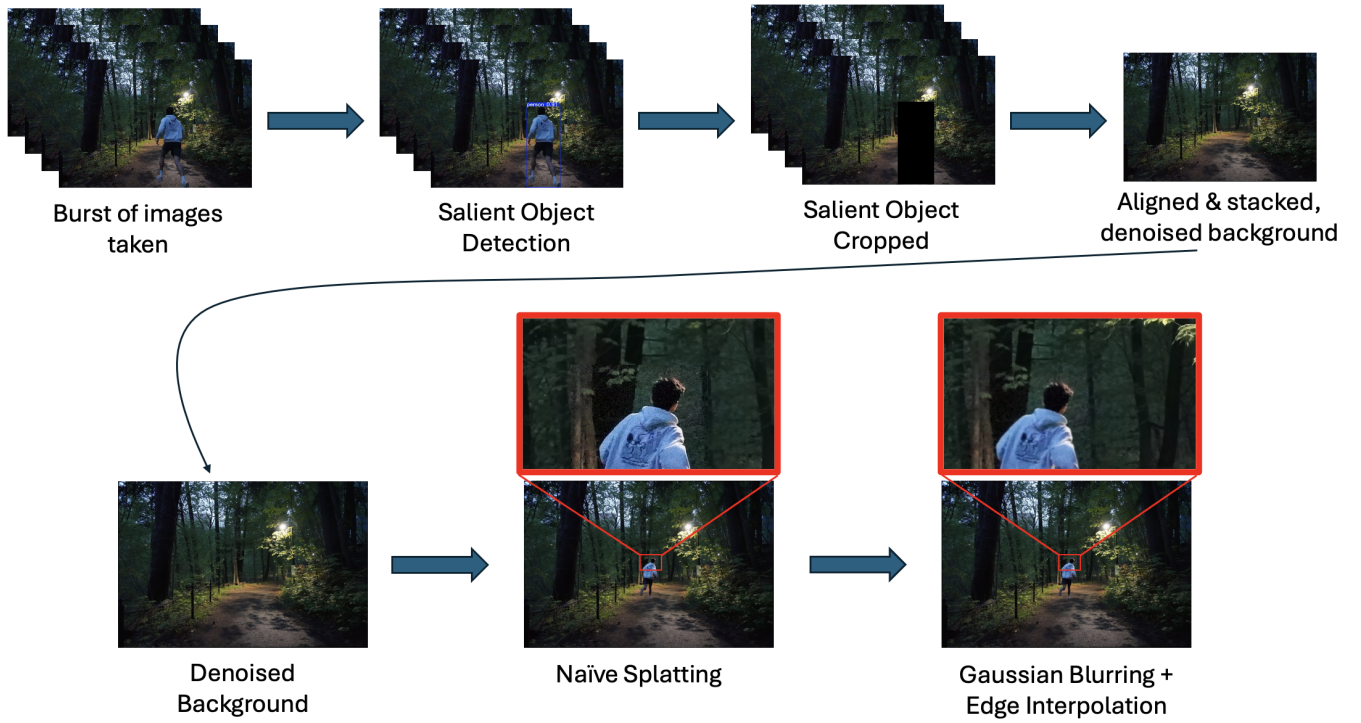


Figure 5: Pipeline for burst processing, with burst image sequence as input. First row demonstrates background denoising with salient object detection and removal. The second (optional) row showcases how you can splat a salient object back onto the scene.

cropped out region x'_j , and replace $\tilde{x}$ with x'_j at the coordinates where we expect x'_j to be.

This still leaves us with a couple of problems. The background of $\tilde{x}$ now has been completely denoised, while x'_j is still noisy. Since the object was moving throughout the frames, we do not have more samples of the object that we can average across¹. Additionally, the edge that goes from $\tilde{x}$ to x'_j is still very noticeable. To reduce noise, we took a 2D gaussian convolution over x'_j to denoise the region. We then interpolated the pixel values along the edges of x'_j with that from $\tilde{x}$ to get rid of the edge artifacts.

3.3 Deblurring System

In the previous sections, we discussed the algorithms used to achieve the desired editing results based on photographer input. In this section, we describe how these algorithms are integrated into a user-friendly interface that allows users to intuitively explore and apply various combinations of editing steps to different salient objects within a scene. A screenshot of our system can be seen in Figure 6.²

To use our system, you first should take a burst of images, and individually save the images to a directory on your computer. At the bottom of your screen, you can specify the burst directory which contains images from the burst. The images can be individually

viewed in the burst timeline, and should be automatically ordered in time (assuming the names of your images are also sorted by time).

On the right hand side is our salient object control panel. This panel displays all the salient objects that have been automatically detected by our salient object detector. By default, our system is set to blur the image to begin with, as outlined in Section ?? . There are also options to remove the salient object, as outlined in section 3.1, or to freeze the object at a specific frame, described in section 3.2. Selecting remove removes all blurring and ghosting artifacts that may be caused by the object.

Freezing opens up an additional slider, labeled “Adjust Frozen Object Pose” in the second salient object in figure 6. This allows you to slide through the positions and poses your salient object is at, and see which position looks the best for the final result. This gives the user the ability to explore different possible locations for the salient object in real time. The details for this feature is outlined in section 3.2.

4 Results

In this section, we will showcase results that our system produces when editing bursts of images, and evaluate it’s performance with respect to certain parameters.

¹Technically if you assume the features of the object remain constant throughout the burst, you do have enough information to denoise, which we talk about in Section 5

²This system was built using PySide6.

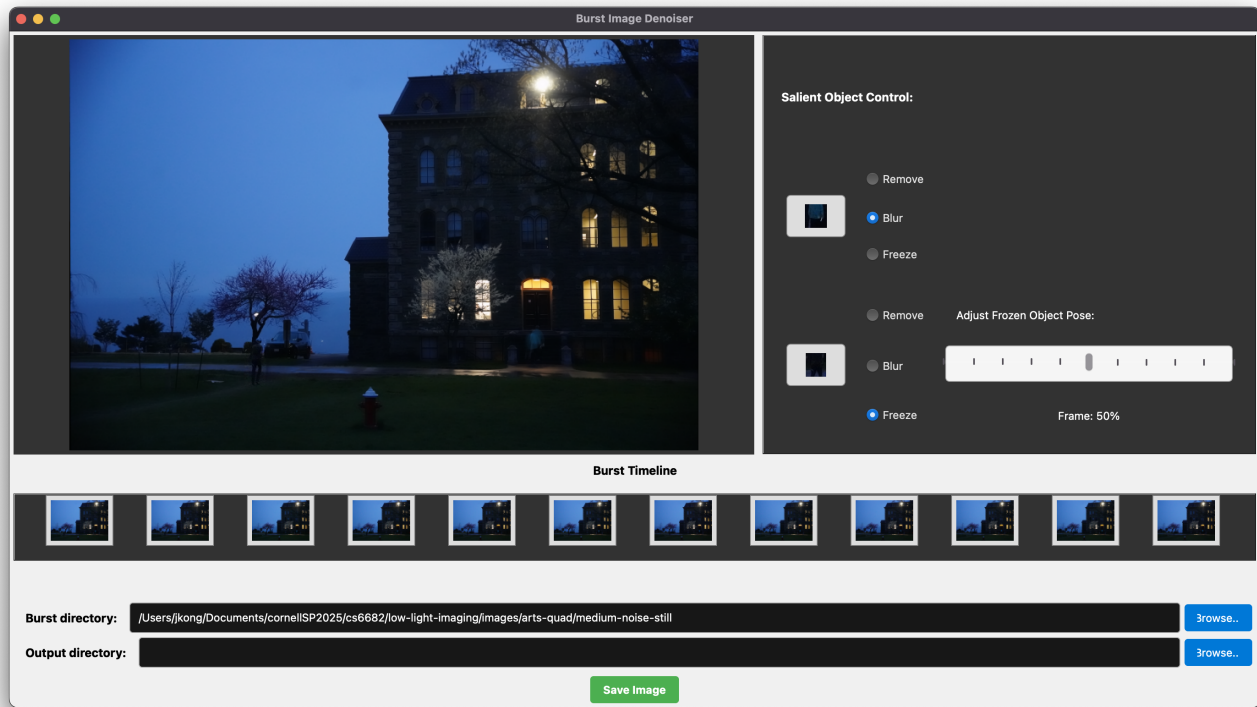


Figure 6: Screenshot of the UI for our editing suite.

4.1 Expressivity

The primary objective of our work was to restore creative control to smartphone photographers over how they could edit low-light bursts. We met this objective in two ways:

4.1.1 Visual Modification of Salient Objects. Our pipeline and editing suite enables users to independently control whether each moving object in the scene should appear as frozen, blurred, or removed.

Figure 1 on the cover of this paper illustrates an example scene of Cornell’s Arts Quad with two people walking over it. Our pipeline recognizes the people as salient, then collecting temporal information about each person for the user to play with. As a result, the user can choose to freeze, blur, or remove each subject from the frame independently. Existing low-light burst photography pipelines would not allow such control, instead pre-rendering the walking people as motion blur.

Such visual modification can be helpful for applications such as architecture photography. Returning to Figure 1, it is plausible that the photographer was taking a low-light burst in order to capture a high-fidelity picture of the building for a future renovation. During the burst exposure, students from campus entered frame, causing motion blur. The photographer can decide that the students distract from the architectural subject and remove them. Conversely, the photographer can choose to freeze the students in frame, determining that they provide a lively foreground for the building. In either case, our editing suite enables the creative freedom to decide.

4.1.2 Temporal Modification of Salient Objects. Our pipeline and editing suite allows users to independently snap each moving object in the scene to any pose they assumed during the duration of the burst. In addition to freezing a moving object in place, our editing

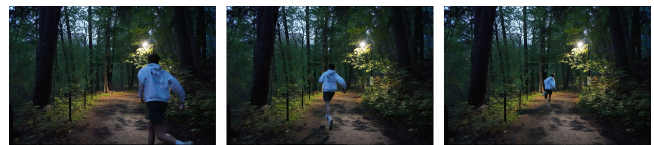


Figure 7: One burst, several key moments. An example of how the user can temporally adjust the position of moving objects in their scene; in this case, the person.

suite allows the user to select the pose from the burst that they want each moving object to assume. Figure 10 shows the applicability of this feature towards action photography. While capturing a runner mid-stride would be impossible with traditional low-light burst imaging, our pipeline enables users to composite still action frames atop a denoised background. This feature enables low-light sports photography on a smartphone, despite the small aperture of a smartphone lens.

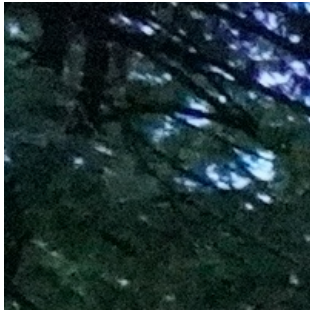


Figure 8: *Noisy Image from frame of burst.*



Figure 9: *Denoised Image from burst processing.*

4.2 Denoising

A large part of why many camera systems opt for burst photography techniques is so that they can have good denoising results. For our background, our pipeline looks to sample many different frames of the background and averages across frames in order to denoise the image.

Figure 8 and 9 are the same region of the same image. Figure 8 was taken from a random frame in the burst of images, while Figure 9 was taken from the resulting image after performing burst processing. It's clear from this image that figure 8 is more granular than figure 9, which has a more constant color across similar regions of the image, and no longer has the granularities that is seen in figure 8.

4.3 Splatting

When it comes to denoising salient objects, especially salient objects that have been splatted onto the background, we unfortunately do not have many images to sample from. This is because tracking features on salient objects is more difficult than tracking features on backgrounds. For example, in the case of a person running, the person moves dynamically and certain areas of the person may not be visible depending on their movements.

To compensate for this, we apply local denoising methods on a frame of the salient object. As seen in the last row of figure 5, the naive splatting method has two problems:

- (1) The noise level in the background of the salient region is much higher than it's surrounding.
- (2) The pixels that boarder the salient region and the background is very noticeable.

To compensate for this, we apply a Gaussian convolution over our salient figure with a kernel size of 5×5 , and $\sigma = 2$. The choice of these parameters depend on the image, and that's what we found worked well for this particular image. To smoothen out the edges, took the 16 outer most pixels, and interpolated its pixel value to be a linear combination of the salient pixel value and the background pixel value. This results in an image that, to the naked eye, shares similar SNR between the salient region and the background.

4.4 Performance

We consider the performance of our system on a baseline 10-image burst of 24-megapixel photographs containing one salient object

each. To load, align, segment, and stack each image in the burst takes our pipeline 12083 milliseconds when run from a Python interpreter and 5220 milliseconds when run from a compiled C++ binary (tested on a single core of an Apple M2 Pro CPU). When run on a 10-image burst of 12-megapixel photographs, our pipeline takes less than three seconds to run; HDR+ takes 4 seconds to process a similar 12-megapixel burst dataset.

Given the different processors used to conduct pipeline testing, we cannot draw direct comparison to HDR+. However, our pipeline remains in the same order of magnitude as established burst imaging pipelines despite the introduction of the YOLO neural segmentation model.

Across the total runtime of our pipeline, we measured that image loading was responsible for 15.5% of runtime, image alignment was responsible for 40.2% of runtime, segmentation was responsible for 10.5%, and image stacking was responsible for the remaining 33.8%. Remarkably, despite computing a forward pass through a CNN, YOLO-driven segmentation was computationally the cheapest part of our pipeline.

Figure 10 plots the effect of salient subjects, image resolution, and burst size on the runtime of each stage of the pipeline. We found that YOLO image segmentation was remarkably robust, and was not affected by either the number of salient objects in a scene nor the resolution of the images. This result makes sense given that images must be compressed to 320x320 before being fed into the segmenting CNN, and only one pass is computed per frame no matter its complexity.

In all, the YOLO salient object detection stage of our pipeline averaged 60 milliseconds per image, or 17Hz. Given the remarkable speed of our salient object detection processing with respect to the load, align, and stack parts of our pipeline, we are interested in integrating our segmentation process with optimized image handling and stacking processes from other works.

5 Discussion

In this paper, we have discussed the limitations of current systems in post-processing burst photos affected by motion blur. Specifically, we introduced methodologies for denoising images while maintaining awareness of salient objects, ensuring that critical details are preserved even in the presence of motion artifacts. We also automated the salient object detection step by utilizing YOLO models to specify regions of interests to photo editors, and showed that integrating real-time segmentation models does not increase runtime compared to established burst imaging pipelines. Finally, we developed a platform from which editors can easily explore different arrangements of output images, turning a previously tedious editing task into an exploratory journey of editing. Our work improves the flexibility and creative expressivity of photo editors, while still maintaining accuracy and precision of the final result.

5.1 Future Work

Our contribution lays the framework for exploratory photo editing of low light images taken with burst photography, and leaves the door open for further improvements and additional features into our system.

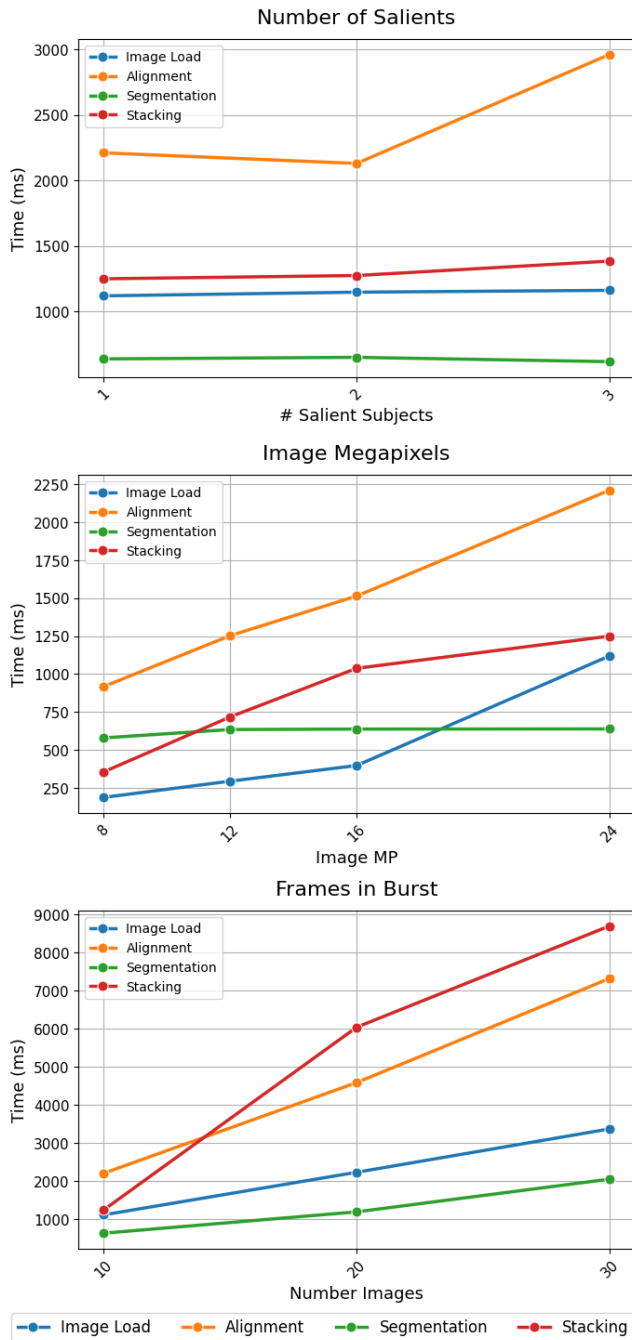


Figure 10: The effect of number of salient objects, image resolution, and burst length of the runtime of each stage of our burst stacking pipeline.

5.1.1 Segmentation Models. Currently we are utilizing YOLO models, which return a bounding box of where the object lies in. The most direct room for improvement is to incorporate a segmentation model so that you can get a more precise outline of the salient object. This allows you to get more samples for certain regions of

the background that will no longer be cropped out by the bounding boxes provided by YOLO, and allows for salient object splatting to have less artifacts. This may also mean that a denoising kernel may not be necessary to do background blending.

5.1.2 Salient Object Sampling. At the moment we are only taking one sample of the salient object and splatting that onto the background. To compensate for the noise in the salient object, we have to apply a convolution to denoise the image. Future work may include averaging across multiple samples of your salient object to perform denoising more informed denoising on the salient object.

5.1.3 Expanding Detectable Objects. The YOLO model has a finite number of classes it can detect, making it limited in application. One of the greatest features of the YOLO model class is that it is easily fine tuneable. For a given set of burst images, it may be possible to only have to draw bounding boxes on a handful of frames over your salient object, and train the model to detect that object throughout your burst (it may even be okay to overfit your model to detect that object). This can expand the number of domains this framework can be used in, and generalize to all objects.

5.1.4 Integration with Existing Burst Alignment Pipelines. Given the remarkable speed of our salient object detection processing with respect to the slow load, align, and stack parts of our pipeline, we are curious to study the performance of a pipeline that leverages our real-time segmentation model alongside existing runtime-optimized burst imaging pipelines.

References

- [1] Paul E. Debevec and Jitendra Malik. 2023. *Recovering High Dynamic Range Radiance Maps from Photographs* (1 ed.). Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3596711.3596779>
- [2] Liyuan Fan, Fan Zhang, Hongbin Fan, and Caiming Wang. 2019. Brief review of image denoising techniques. *Visual Computing for Industry, Biomedicine, and Art* 2, 1 (2019), 7. doi:10.1186/s42492-019-0016-7
- [3] Martin A. Fischler and Robert C. Bolles. 1981. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Commun. ACM* 24, 6 (June 1981), 381–395. doi:10.1145/358669.358692
- [4] Clément Godard, Kevin Matzen, and Matt Uyttendaele. 2017. Deep Burst Denoising. *CoRR abs/1712.05790* (2017). arXiv:1712.05790 <http://arxiv.org/abs/1712.05790>
- [5] Clément Godard, Kevin Matzen, and Matt Uyttendaele. 2018. Deep Burst Denoising. In *Computer Vision – ECCV 2018: 15th European Conference, Munich, Germany, September 8–14, 2018, Proceedings, Part XV* (Munich, Germany). Springer-Verlag, Berlin, Heidelberg, 560–577. doi:10.1007/978-3-030-01267-0_33
- [6] Sam Hasinoff, Dillon Sharlet, Ryan Geiss, Andrew Adams, Jonathan T. Barron, Florian Kainz, Jiawen Chen, and Marc Levoy. 2016. Burst photography for high dynamic range and low-light imaging on mobile cameras. *SIGGRAPH Asia* (2016). <http://www.hdrplusdata.org/hdrplus.pdf>
- [7] Atsushi Ito and Aswin C. Sankaranarayanan. 2013. BlurBurst : Removing Blur Due to Camera Shake using Multiple Images. <https://api.semanticscholar.org/CorpusID:3809610>
- [8] Nidhal Jegham, Chan Young Koh, Marwan Abdelatti, and Abdeltawab Hendawi. 2024. Evaluating the Evolution of YOLO (You Only Look Once) Models: A Comprehensive Benchmark Study of YOLO11 and Its Predecessors. doi:10.48550/arXiv.2411.00201
- [9] Tony Lindeberg. 2012. *Scale Invariant Feature Transform*. Vol. 7. doi:10.4249/scholarpedia.10491
- [10] Joseph Redmon, Santosh Kumar Divvala, Ross B. Girshick, and Ali Farhadi. 2015. You Only Look Once: Unified, Real-Time Object Detection. *CoRR abs/1506.02640* (2015). arXiv:1506.02640 <http://arxiv.org/abs/1506.02640>
- [11] Eric Tabellion, Nikhil Karnad, Noa Glaser, Ben Weiss, David E. Jacobs, and Yael Pritch. 2023. Computational Long Exposure Mobile Photography. *ACM Trans. Graph.* 42, 4, Article 48 (July 2023), 15 pages. doi:10.1145/3592124